

LLD Practice Platform

README — Engineering Assignment Prototype

1. Overview

The LLD Practice Platform is a focused practice experience for learners to solve Low-Level Design problems, submit their design, receive structured feedback, review previous attempts, and retry. The MVP follows a simple loop: **Choose Problem** → **Practice** → **Submit** → **Evaluate** → **Review** → **Retry**.

The platform is intentionally designed as a practical modular monolith. The core goal is to demonstrate domain design, evaluation thinking, explainable feedback, reliability, and extensibility without adding unnecessary distributed-system complexity.

2. MVP Features

- **Problem selection:** three seeded LLD problems with title, difficulty, description, requirements, and constraints.
- **Practice attempt:** learners create an attempt and provide their approach, classes, relationships, abstractions, patterns, assumptions, and edge cases.
- **Draft support:** an in-progress submission can be saved before final submission.
- **Evaluation:** deterministic input validation is followed by structured LLM-based qualitative evaluation.
- **Explainable feedback:** each rubric criterion includes score, evidence, concern, suggestion, and confidence.
- **Evaluation states:** Submitted → Evaluating → Completed / Failed.
- **History and retry:** previous attempts can be reviewed, failed evaluations can be retried, and completed work can be revisited.

3. Tech Stack

Layer	Technology
Frontend	React.js + Vite + plain CSS + React Router
Backend	Node.js + Express.js
Database	MongoDB + Mongoose
AI Evaluation	Google Gemini via @google/genai
Testing	Node.js built-in node:test

4. Current Project Structure

```
lld-practice-platform/  
  
  • client/ - React frontend  
  • server/ - Express backend and domain logic  
  • docs/RESEARCH.md - research note  
  • docs/DESIGN.md - design note  
  • AI_USAGE.md - AI-assisted development report  
  • README.md - project documentation  
  • .gitignore / server/.env.example
```

5. Architecture & Domain Model

The backend is organized as a modular monolith. HTTP controllers handle requests and responses, models persist domain data, and services contain evaluation logic. The evaluator abstraction keeps the practice flow independent from a particular evaluation technology.

Entity	Responsibility
Problem	Stores the LLD challenge, requirements, difficulty, description, and constraints.
Attempt	Represents a learner's practice session and its lifecycle/status.
Submission	Stores the learner's design response associated with one attempt.
Evaluation	Stores evaluator type, status, rubric criteria, feedback, strengths, improvements, and errors.
Evaluator	Abstract contract for evaluating a problem/submission against the rubric.

Relationships

- One **Problem** can have many **Attempts**.
- One **Attempt** has one **Submission**.
- One **Attempt** has one **Evaluation**.
- An **Evaluation** records which evaluator produced the result.

6. Evaluation Approach

The platform separates objective validation from qualitative design judgment. Required submission sections are validated before evaluation. The LLM then evaluates the design using a fixed eight-criterion rubric and returns a structured response.

Rubric

- Requirement Understanding
- Class Responsibilities
- Coupling & Cohesion
- Encapsulation & Interfaces
- Abstraction & Patterns
- Extensibility
- Edge Cases & Testability
- Explanation Quality

Each criterion produces a 1–5 score plus evidence, concern, suggestion, and confidence. The evaluator response is validated before it is persisted, preventing malformed AI output from silently becoming application data.

7. Evaluation State & Reliability

The submission is persisted before evaluation begins. This avoids losing learner work if the external model is slow or unavailable. The attempt lifecycle is explicit: **DRAFT** → **SUBMITTED** → **EVALUATING** → **COMPLETED** / **FAILED**.

- **Submitted**: learner has finalized the submission.
- **Evaluating**: evaluation is in progress.
- **Completed**: valid evaluation has been stored and can be reviewed.
- **Failed**: evaluation could not complete; the learner can retry without rewriting the submission.
- **Retry**: the existing persisted submission is evaluated again, rather than forcing the learner to submit it again.

8. Multiple Valid Solutions

LLD problems rarely have only one correct class structure. The evaluator therefore does not compare the submission against a single hard-coded reference answer. Instead, it judges the design against stable principles such as responsibility boundaries, cohesion, coupling, abstraction, extensibility, edge cases, and explanation quality.

9. Extensibility

The design intentionally isolates evaluation behind an **Evaluator** abstraction. This supports two important future changes without rewriting the practice flow:

- **Change A — Submission format:** move from text-based design input toward a class diagram or another structured representation while keeping the core Problem/Attempt/Submission/Evaluation model stable.
- **Change B — Evaluator implementation:** replace or supplement the LLM evaluator with a rule-based or human evaluator through the same evaluation boundary.

10. Persistence

MongoDB stores problems, attempts, submissions, and evaluations. An attempt references its problem, while submission and evaluation reference the attempt. This keeps learner history durable and allows the evaluation result to be reviewed independently of the live evaluator.

11. Running the Project

Prerequisites: Node.js, MongoDB, and a Gemini API key for LLM evaluation.

Backend

```
cd server
npm install
npm run seed
npm run dev
```

Configure **server/.env** using **server/.env.example**: PORT, MONGO_URI, and GEMINI_API_KEY.

Frontend

```
cd client
npm install
npm run dev
```

The frontend calls the backend API at the configured local API base URL.

12. Testing

Backend tests use Node's built-in test runner. The current test suite covers evaluation input validation and structured evaluation-response validation, including valid responses, missing rubric criteria, invalid scores, invalid confidence values, and incorrect criterion counts.

Run tests with:

```
cd server
npm test
```

13. Scope, Trade-offs & Limitations

The assignment is intentionally scoped around the learner practice loop and LLD/domain design. The implementation avoids infrastructure complexity that does not materially improve the MVP.

- **Included:** problem selection, attempt creation, draft saving, structured submission, deterministic validation, LLM evaluation, explainable feedback, history, retry, persistence, and tests.
- **Not required for the MVP:** authentication, payments, social features, large-scale microservices, Kubernetes, multi-region deployment, sharding, CDN infrastructure, Kafka pipelines, or Redis-based distributed state.
- **Trade-off:** a modular monolith is simpler to build, understand, test, and evolve for this scale while retaining clear boundaries for future extraction.

- **Limitation:** LLM evaluation is inherently probabilistic and depends on model availability and output quality; response validation and explicit failure states reduce, but do not eliminate, this risk.

14. Future Improvements

- Authentication and learner profiles.
- More LLD problem categories and difficulty levels.
- Structured diagram/class-model submission.
- Reference examples and guided hints.
- Evaluator versioning and evaluation analytics.
- Human review or rule-based evaluator alongside the LLM evaluator.
- Asynchronous evaluation infrastructure if usage grows significantly.

15. AI Usage

AI was used as a development assistant rather than as the author of the application. Meaningful AI-assisted decisions included rubric design, evaluator abstraction, structured LLM output, deterministic validation combined with qualitative evaluation, and evaluation failure handling.

The final architecture, implementation, MongoDB models, API flow, Gemini integration, frontend, tests, documentation, and trade-offs were reviewed and owned by the developer. AI suggestions were accepted selectively when they improved clarity or reliability and rejected when they added unnecessary complexity.

See **AI_USAGE.md** for the detailed record of AI-assisted decisions, accepted/rejected suggestions, coding/debugging assistance, and what was deliberately not delegated to AI.

16. Documentation

- **docs/RESEARCH.md:** problem understanding, existing approaches, observed gaps, product direction, and MVP scope.
- **docs/DESIGN.md:** MVP flow, domain model, evaluation design, state handling, extensibility, architecture, persistence, trade-offs, and limitations.
- **AI_USAGE.md:** transparent AI-assisted development report.

17. Current MVP Flow

Home → Choose an LLD Problem → Start Practice → Fill Design Sections → Save Draft / Submit → Evaluate → View Structured Feedback → History → Review or Retry

18. Engineering Principle

Keep the learner flow simple while keeping domain boundaries explicit. The platform is designed so that the practice experience does not depend on one evaluator, one submission format, or a perfect external AI response.

Author: Naman Jain

AI_USAGE.md

AI-Assisted Development Report — LLD Practice Platform

1. Purpose

AI was used as a development assistant throughout the LLD Practice Platform assignment. It was used selectively for research, brainstorming, debugging, validation, reviewing, and documentation. The core product direction, architecture, implementation, and engineering trade-offs remained the developer's responsibility.

The goal of using AI was to accelerate iteration and challenge design decisions, not to delegate ownership of the application.

2. Meaningful AI-Assisted Decisions

Decision 1 — Evaluation Rubric Design

What AI suggested: A structured rubric would make LLD feedback more consistent and useful than a single generic score.

What was accepted: An eight-criterion rubric covering Requirement Understanding, Class Responsibilities, Coupling & Cohesion, Encapsulation & Interfaces, Abstraction & Patterns, Extensibility, Edge Cases & Testability, and Explanation Quality.

Why: These criteria directly support the assignment's requirement for explainable feedback while allowing multiple valid LLD solutions.

Decision 2 — Evaluator Abstraction

What AI suggested: Separate the evaluator contract from the concrete evaluation implementation.

What was accepted: An **Evaluator** abstraction with a concrete Gemini-based evaluator.

Why: The practice flow should not depend permanently on one evaluation mechanism. This also supports future rule-based or human evaluation.

Decision 3 — Structured LLM Evaluation

What AI suggested: Ask the model for structured output instead of parsing free-form feedback.

What was accepted: The evaluator returns criterion-level score, evidence, concern, suggestion, and confidence, along with overall feedback, strengths, and improvements.

Why: Structured output is easier to validate, persist, display, and evolve.

3. Additional AI-Assisted Decisions

Decision 4 — Deterministic Validation + LLM Evaluation

What AI suggested: Do not rely entirely on the LLM for basic validity checks.

What was accepted: Required submission sections are validated deterministically before qualitative LLM evaluation. The generated evaluation response is also validated before persistence.

Why: Basic application rules should be predictable, while subjective LLD judgment is where an LLM provides value.

Decision 5 — Evaluation Failure Handling

What AI suggested: Treat evaluation as a stateful operation and preserve the learner's submission when an external model fails.

What was accepted: The attempt moves through **Submitted** → **Evaluating** → **Completed / Failed**. The submission is persisted before evaluation, and a failed evaluation can be retried without submitting the work again.

Why: External AI services can be slow or temporarily unavailable. Learner work should not be lost, and failure should be recoverable.

4. AI-Assisted Coding & Debugging

AI was also used during implementation to help reason about code structure, diagnose errors, review API behavior, and refine documentation. This included debugging backend integration issues and checking validation/evaluation flows.

The development process remained iterative: AI suggestions were inspected against the actual codebase, tested locally, and changed when they did not fit the project's scope or requirements.

Examples of development assistance included:

- Reasoning about controller/service responsibilities and keeping evaluation logic separated.
- Reviewing validation rules and structured evaluation response handling.
- Helping diagnose external Gemini API/model availability issues and integration errors.
- Reviewing failure and retry behavior after an evaluation service error.
- Improving README, design documentation, and explanation of engineering trade-offs.

5. What Was Accepted vs. Rejected

AI direction	Decision	Reason
Structured rubric	Accepted	Improves consistency and explainability.
Evaluator abstraction	Accepted	Supports future evaluator implementations.
Structured JSON output	Accepted	Reliable validation and UI rendering.
Deterministic validation before LLM	Accepted	Keeps objective rules predictable.
Stateful failure + retry	Accepted	Protects learner work and improves reliability.
Unnecessary distributed infrastructure	Rejected	Outside MVP scope and adds complexity without enough value.

6. What I Did Not Delegate to AI

AI did not independently define or own the final application. The following areas were deliberately kept under developer control:

- Final product scope and the learner practice flow.
- Final domain model and MongoDB schema decisions.
- API structure, controller behavior, and persistence flow.
- Frontend page flow and implementation.
- Gemini integration and handling of real API failures.
- Test creation, execution, and interpretation of test results.
- Final architecture, trade-offs, limitations, and extensibility decisions.
- Final review of generated code and documentation before inclusion.
-

7. Engineering Ownership

AI suggestions were treated as proposals rather than authoritative answers. The developer evaluated whether each suggestion was appropriate for the assignment, implemented selected ideas, tested them, and rejected ideas that introduced unnecessary complexity.

This was especially important because the assignment explicitly values engineering judgment. For example, the project remains a modular monolith rather than introducing Kafka, Redis, Kubernetes, microservices, or other distributed infrastructure that was not necessary for the MVP.

8. Result of AI-Assisted Development

- A fixed and explainable evaluation rubric was established.
- Evaluation became replaceable through an evaluator abstraction.
- LLM output became structured and validated before persistence.
- Learner submissions are protected from temporary evaluator failures.
- The platform supports retrying failed evaluations.
- The documentation clearly separates product/domain decisions from AI-assisted suggestions.
-

9. Summary

AI accelerated research, brainstorming, debugging, validation, and documentation, but the final solution was reviewed and owned by the developer. The most valuable use of AI was not generating large amounts of code; it was helping challenge design decisions and identify practical reliability and extensibility patterns.

The resulting implementation intentionally balances AI capability with deterministic application behavior: objective validation remains predictable, while the LLM is used where qualitative LLD judgment is useful.

Developer: Naman Jain

Project: LLD Practice Platform — Engineering Assignment